- A1 will be posted later today
- Start with reminder about $fl(x+y)$ vs $x+y$.

Fri 17 Jan 2025

1

Computing an arithmetic result with error $\le \varepsilon_{mach}$ is the best that we can expect since we may need to make a representation error when representing the computed result in F.

Not all processors meet this guarantee.

For IEEE arithmetic, when $x, y \in F$, and $op \in \{+, -, *, /\}$, $\dfrac{|fl(x \text{ op } y) - (x \text{ op } y)|}{|x \text{ op } y|}$ must be $\le \varepsilon_{mach}$

also $\sqrt{\ }$ $\quad \dfrac{|fl(\text{sqrt}(x)) - \sqrt{x}|}{|\sqrt{x}|} \le \varepsilon_{mach}$

We must get the correctly rounded result.

Multiple operations

The above tells you about the error in a single operation $(x \text{ op } y)$ with exact data. What if you have multiple operations? How can errors compine?

Suppose we have two calculations

```
expr1 = sqrt(2.0) + log(42.0)
expr2 = exp(7.0) + sin(45.0)
```

expr1, expr2 > 0.

Then $\varepsilon_{expr1} = \dfrac{|\overset{\text{computed result}}{expr1} - (\sqrt{2} + \ln(42))|}{|\underbrace{\sqrt{2} + \ln(42)}_{\text{exact math value}}|}$

and similarly for ε_{expr2}

Now suppose we compute

expr1 + expr2

expr1 * expr2

expr1 / expr2

Question: How do the previous errors accumulate?

You can show that when IEEE arithmetic is used; the relative errors satisfy:

$$\varepsilon_{expr1 + expr2} \leq \max(\varepsilon_{expr1}, \varepsilon_{expr2}) + |\varepsilon_A|$$

the error from representing sum

$$\varepsilon_{expr1 * expr2} \leq \varepsilon_{expr1} + \varepsilon_{expr2} + |\varepsilon_B|$$

$$\varepsilon_{expr1 / expr2} \leq \underbrace{\varepsilon_{expr1} + \varepsilon_{expr2}}_{\text{error due to inexact operands}} + \underbrace{|\varepsilon_C|}_{\text{error due to representing result.}}$$

$$|\varepsilon_A|, |\varepsilon_B|, |\varepsilon_C| \leq \varepsilon_{mach}$$

Takeaways:

- the error does not grow <u>too</u> quickly!
- we did not mention subtraction!

How do errors accumulate when subtraction is performed?

Suppose we want to compute the roots of the quadratic equation

$$x^2 + (-320)x + 16 = 0$$

In high school, you learned that the roots of $ax^2 + bx + c = 0$ satisfy

$$r_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}.$$

Using this rule with $F(10, 4, L, u)$ we get

<see the posted sample-calculations.pdf for a tutorial example>

$$r_1 = \frac{3.200 \times 10^2 + \sqrt{1.024 \times 10^5 - 6.400 \times 10^1}}{2.000 \times 10^0}$$

$$\doteq \frac{3.200 \times 10^2 + \sqrt{1.023 \times 10^5}}{2.000 \times 10^0}$$ ← a rounding error introduced

$$\doteq \frac{3.200 \times 10^2 + 3.198 \times 10^2}{2.000 \times 10^0}$$ ← another rounding error introduced

$$= \frac{6.398 \times 10^2}{2.000 \times 10^0}$$

$$= 3.199 \times 10^2$$

and then $$r_2 \doteq \frac{3.200 \times 10^2 - 3.198 \times 10^2}{2.000 \times 10^0}$$

$$= \frac{2.000 \times 10^{-1}}{2.000 \times 10^0}$$

$$= 1.000 \times 10^{-1}$$

You can show that the exact roots are

$r_1^* \doteq 319.950$ $\quad r_2^* \doteq 0.05001$

so rel error $\varepsilon_{r_1} \doteq 1.6 \times 10^{-4} < \varepsilon_{mach} = 5 \times 10^{-4}$ $\quad \varepsilon_{r_2} \approx 1.00$ means NO accurate digits!

What went wrong?! We calculated

$$r_2 = \frac{\overset{\text{exact}}{3.200 \times 10^2} - 3.198 \times 10^2}{\underset{\text{exact}}{2.000 \times 10^0}}$$

$fl\left(\sqrt{fl(102400-64)}\right)$

contains a couple of round errors.

In the subtraction, we get 0.002×10^2 but what we really need is to subtract

$3.200 \times 10^2 - 3.198\,xxxx \times 10^2$

The subtraction removes or cancels most of the significant digits. Unfortunately, the lost digits in the accurate (but inexact) intermediate result $3.198\,xxxx \times 10^2$ are now significant in the desired result. But we have no idea what they are!

This phenomenon is called <u>catastrophic cancellation</u>

It can occur whenever you subtract two (possibly) inexact numbers of similar size.

The subtraction reveals the occurrence and importance of an earlier approximation error

$\therefore$ when working with floating point numbers, try to design your software to avoid the subtraction of nearly equal values, when possible.

$\boxed{\text{How}}$ can we accurately compute 'the other root' to our equation?

Proposal ① compute using a higher precision
→ won't always work since can have same issue at higher precision for same numbers

② observe
Since $(1x^2 + -320x + 16)$
can be factored into $1\cdot(x-r_1)(x-r_2)$,

by matching the constant terms we have

$$1\cdot r_1 r_2 = 16 \quad \text{and} \quad r_2 = 16/(r_1\cdot 1)$$

or $$r_2 = \frac{1.600\times 10^1}{3.199\times 10^2}$$
$$= 5.002\times 10^{-2}$$

we have $$\varepsilon_{r_2} = \frac{|r_2 - r_2^*|}{|r_2^*|}$$
$$= 2.4\times 10^{-4}$$
$$< \varepsilon_{mach} \quad 5\times 10^{-4}$$

So we now have r_2 to good accuracy!

$\therefore$ a better algorithm for computing the roots of $ax^2 + bx + c = 0$ is to take

$$r_1 = \frac{-b - \text{sign}(b)\sqrt{b^2 - 4ac}}{2a}$$

$$r_2 = c/(a r_1)$$

where $\text{sign}(b) = \begin{cases} +1, & b \geq 0 \\ -1, & b < 0. \end{cases}$

It may seem inevitable that your algorithms will contain catastrophic cancellations. Sometimes you can use ingenuity to avoid them.

To be continued …